

AI ASSISTENT ASSIGNMENT - 19.3

Baisa Vaishnavi

2303A52142

Batch - 41

Task 1 – Python to C++ Conversion

Task:

Translate a simple Python class into C++ using AI assistance.

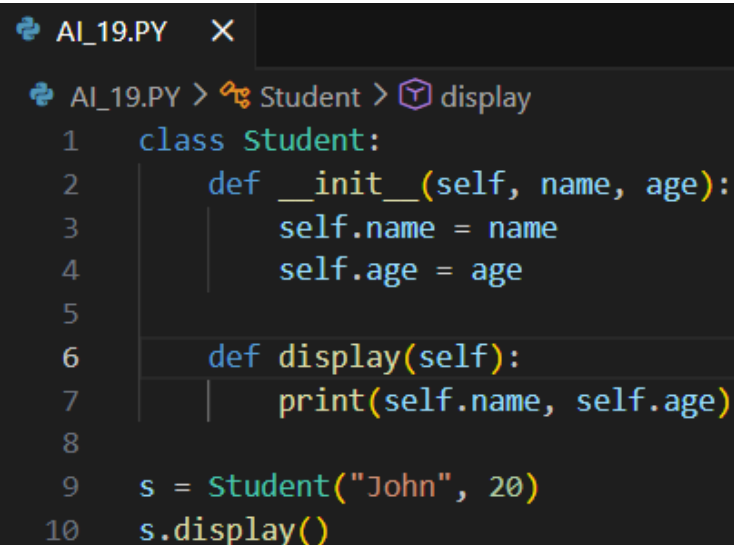
Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
 - o Constructors
 - o Data types
 - o Access specifiers
- Compile and run both versions to verify output consistency

Prompt

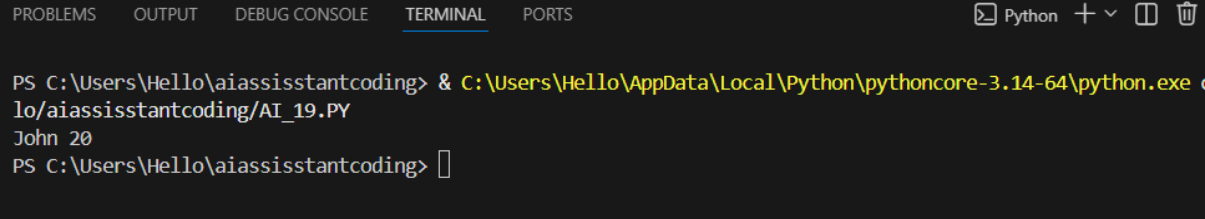
Convert a Python class Student with attributes name, age and a display() method into equivalent C++ code with constructor and proper access specifiers.

Python code



```
AI_19.PY X
AI_19.PY > Student > display
1 class Student:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def display(self):
7         print(self.name, self.age)
8
9 s = Student("John", 20)
10 s.display()
```

Output:



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the following commands and output:

```
PS C:\Users\Hello\aiassisstantcoding> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\Hello\aiassisstantcoding\AI_19.PY
John 20
PS C:\Users\Hello\aiassisstantcoding> 
```

Justification

Constructor added explicitly in C++
Data types defined (string, int)
Encapsulation via private/public
Output verified same in both languages.

Task 2 – Java to Python Function Conversion

Task:

Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Prompt:

Convert Java method isPrime(int n) into Python function using Pythonic syntax.

Python Code

```

AI_19_2.py > ...
1  def is_prime(n):
2      if n <= 1:
3          return False
4
5      for i in range(2, n // 2 + 1):
6          if n % i == 0:
7              return False
8
9      return True
10
11  # Test cases
12  print(is_prime(7))    # True
13  print(is_prime(10))  # False

```

Output:

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ] ... |

PS C:\Users\Hello\aiassisstantcoding> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users\Hello\aiassisstantcoding/AI_19_2.py
True
False
PS C:\Users\Hello\aiassisstantcoding>

```

Java code & output:

```

J PrimeCheck.java > PrimeCheck
1  public class PrimeCheck {
2
3      // Method to check if number is prime
4      public static boolean isPrime(int n) {
5          if (n <= 1)
6              return false;
7
8          for (int i = 2; i <= n / 2; i++) {
9              if (n % i == 0)
10                 return false;
11          }
12
13          return true;
14      }
15
16      // Main method to test
17      public static void main(String[] args) {
18          System.out.println(isPrime(7)); // true
19          System.out.println(isPrime(10)); // false
20      }
21  }

```

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Run: PrimeCheck + - [ ] [ ] ... |

PS C:\Users\Hello\aiassisstantcoding> & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.17-hotspot\bin\java.exe'
'-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Hello\AppData\Roaming\Code\User\workspaceStorage\064f
bd4d9e8851f0237d985f02559017\redhat.java\jdt_ws\aiassisstantcoding_a6079629\bin' 'PrimeCheck'
true
false
PS C:\Users\Hello\aiassisstantcoding>

```

Justification:

Python uses indentation instead of braces

range() replaces loop structure

Cleaner syntax (no type declaration)

Logic preserved exactly

Task 3 – Pseudocode to Python Implementation**Task:**

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

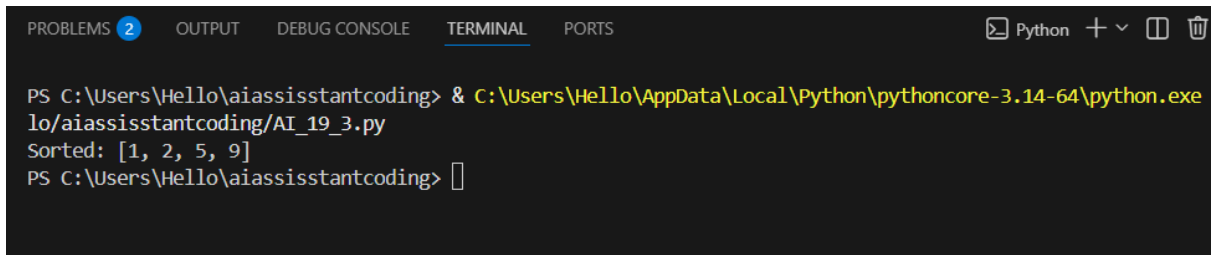
Prompt

Convert bubble sort pseudocode into Python program.

Code:

```
AI_19_3.py > ...
1  def bubble_sort(arr):
2      n = len(arr)
3
4      for i in range(n):
5          for j in range(0, n - i - 1):
6              if arr[j] > arr[j + 1]:
7                  # Swap
8                  arr[j], arr[j + 1] = arr[j + 1], arr[j]
9
10     return arr
11
12     # Test
13     data = [5, 2, 9, 1]
14     print("Sorted:", bubble_sort(data))
```

Output:

A screenshot of a Visual Studio Code terminal window. The terminal has tabs for PROBLEMS (with a blue circle containing the number 2), OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The terminal shows a PowerShell prompt at C:\Users\Hello\aiassistantcoding> where a command to run a Python script is entered. The output of the script is displayed as 'Sorted: [1, 2, 5, 9]'.

```
PS C:\Users\Hello\aiassistantcoding> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe  
lo/aiassistantcoding/AI_19_3.py  
Sorted: [1, 2, 5, 9]  
PS C:\Users\Hello\aiassistantcoding> 
```

Justification:

Pseudocode loops directly mapped to Python loops

Swap done using tuple unpacking

Algorithm correctness verified

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame

Prompt:

Convert SQL query SELECT name, salary FROM employees WHERE salary > 50000 into Pandas code.

Python pandas Code:

```

J AI_19_4.java
1  import pandas as pd
2
3  # Sample data
4  data = {
5      "name": ["A", "B", "C"],
6      "salary": [40000, 60000, 70000]
7  }
8
9  df = pd.DataFrame(data)
10
11 # Equivalent to SQL:
12 # SELECT name, salary FROM employees WHERE salary > 50000
13
14 result = df[df["salary"] > 50000][["name", "salary"]]
15
16 print(result)
17

```

Output:

```

PROBLEMS 22 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Run: PrimeCheck + - [ ] [X]

PS C:\Users\Hello\aiassisstantcoding> & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.17-hotspot\bin\java
+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Hello\AppData\Roaming\Code\User\workspaceStorage\064ft
237d985f02559017\redhat.java\jdt_ws\aiassisstantcoding_a6079629\bin' 'PrimeCheck'
true
false
PS C:\Users\Hello\aiassisstantcoding>

```

Justification:

WHERE → Boolean filtering

SELECT → column selection

Equivalent data subset returned

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

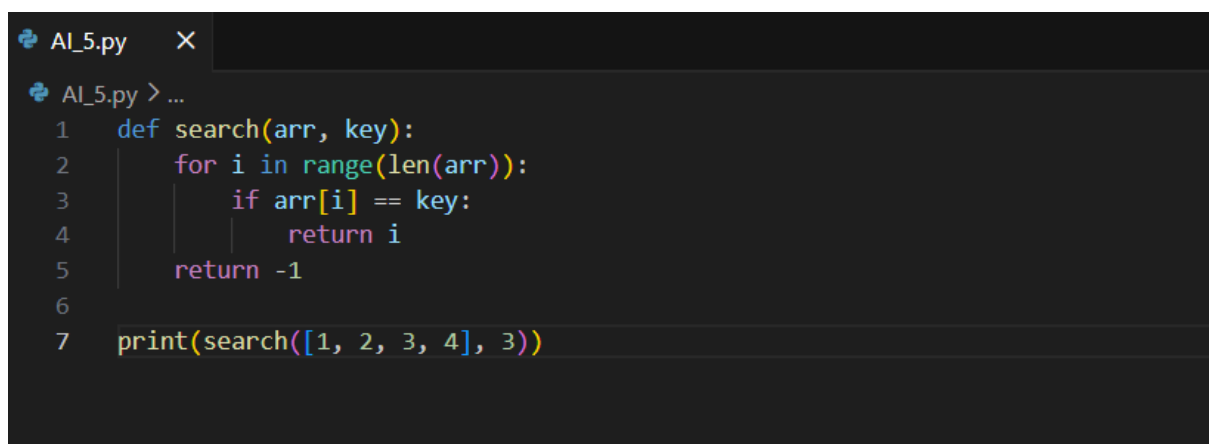
Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - o Syntax differences
 - o Library support
 - o Memory management

Prompt

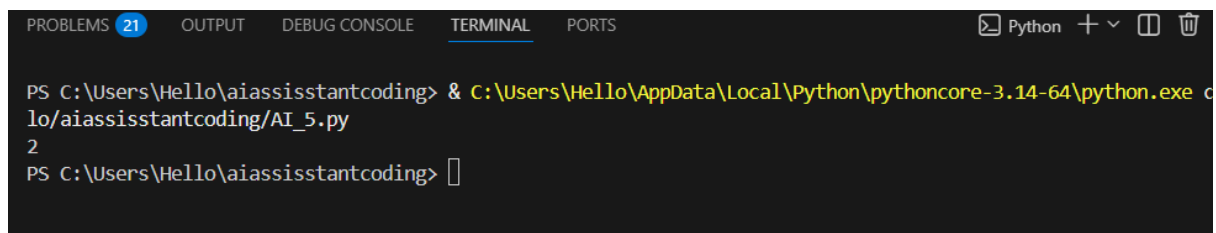
Convert Python linear search algorithm into C language with proper memory handling.

Code:

A screenshot of a code editor window titled 'AI_5.py'. The editor shows a Python function 'search(arr, key)' that iterates through an array 'arr' to find a 'key'. If the key is found, it returns the index 'i'; otherwise, it returns -1. Below the function, there is a print statement: 'print(search([1, 2, 3, 4], 3))'.

```
AI_5.py X
AI_5.py > ...
1 def search(arr, key):
2     for i in range(len(arr)):
3         if arr[i] == key:
4             return i
5     return -1
6
7 print(search([1, 2, 3, 4], 3))
```

Output:

A screenshot of a terminal window with tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing a command prompt where the command to run 'AI_5.py' has been entered. The output of the script is '2'.

```
PROBLEMS 21 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Python + - [ ] [ ]
PS C:\Users\Hello\aiassistantcoding> & C:\Users\Hello\AppData\Local\Python\pythoncore-3.14-64\python.exe c:\Users\Hello\aiassistantcoding\AI_5.py
2
PS C:\Users\Hello\aiassistantcoding> [ ]
```

Justification :

Manual array handling in C
No dynamic typing → explicit sizes required
Memory handled via static arrays
Same logic preserved